

Lecture 9: Normalization and BCNF

Amit Kumar Dhar

August 21, 2025

The Need for Normalization

Why Normalize? A Problematic Schema

Consider the following schema for tracking staff assigned to projects:

staffID	staffName	projectID	projectName
S101	Alice	P50	Project Phoenix
S102	Bob	P75	Project Hydra
S101	Alice	P75	Project Hydra
S103	Charlie	P50	Project Phoenix

Functional Dependencies:

- $\text{staffID} \rightarrow \text{staffName}$
- $\text{projectID} \rightarrow \text{projectName}$

This simple design leads to several anomalies.

Anomalies in the Schema

- **Redundancy:** The name of a staff member (e.g., Alice) and the name of a project (e.g., Project Phoenix) are repeated for every assignment. This wastes space.

Anomalies in the Schema

- **Redundancy:** The name of a staff member (e.g., Alice) and the name of a project (e.g., Project Phoenix) are repeated for every assignment. This wastes space.
- **Update Anomaly:** If Project Phoenix is renamed, we must update it in multiple rows. Forgetting even one leads to inconsistent data.

Anomalies in the Schema

- **Redundancy:** The name of a staff member (e.g., Alice) and the name of a project (e.g., Project Phoenix) are repeated for every assignment. This wastes space.
- **Update Anomaly:** If Project Phoenix is renamed, we must update it in multiple rows. Forgetting even one leads to inconsistent data.
- **Insertion Anomaly:** We cannot add a new staff member until they are assigned to a project. Similarly, we cannot add a new project until at least one staff member is assigned.

Anomalies in the Schema

- **Redundancy:** The name of a staff member (e.g., Alice) and the name of a project (e.g., Project Phoenix) are repeated for every assignment. This wastes space.
- **Update Anomaly:** If Project Phoenix is renamed, we must update it in multiple rows. Forgetting even one leads to inconsistent data.
- **Insertion Anomaly:** We cannot add a new staff member until they are assigned to a project. Similarly, we cannot add a new project until at least one staff member is assigned.
- **Deletion Anomaly:** If Charlie is the only staff member on Project Phoenix and their assignment is deleted, we lose all information about Project Phoenix.

Anomalies in the Schema

- **Redundancy:** The name of a staff member (e.g., Alice) and the name of a project (e.g., Project Phoenix) are repeated for every assignment. This wastes space.
- **Update Anomaly:** If Project Phoenix is renamed, we must update it in multiple rows. Forgetting even one leads to inconsistent data.
- **Insertion Anomaly:** We cannot add a new staff member until they are assigned to a project. Similarly, we cannot add a new project until at least one staff member is assigned.
- **Deletion Anomaly:** If Charlie is the only staff member on Project Phoenix and their assignment is deleted, we lose all information about Project Phoenix.

Solution: Decompose the relation into smaller, well-structured relations.

Decomposition as the Solution

We can break the problematic relation into two separate relations:

Staff_Info

staffID	staffName
S101	Alice
S102	Bob
S103	Charlie

Project_Info

projectID	projectName
P50	Project Phoenix
P75	Project Hydra

Staff_Assignments

staffID	projectID
S101	P50
S102	P75
S101	P75
S103	P50

This decomposition eliminates all the anomalies! Normalization gives us a formal process to achieve this.

Boyce-Codd Normal Form (BCNF)

Introduction to Normal Forms

Normal forms are a series of guidelines to evaluate and improve the design of a relational schema.

- First Normal Form (1NF) - Atomic values
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- **Boyce-Codd Normal Form (BCNF)**
- Fourth Normal Form (4NF)
- Fifth Normal Form (5NF)

Each normal form is stricter than the previous one. BCNF is considered the standard for a "good" relational design.

Boyce-Codd Normal Form (BCNF)

Invented by Raymond F. Boyce and Edgar F. Codd to address certain anomalies not handled by 3NF.

Definition: A relation R is in BCNF if for every non-trivial functional dependency $\alpha \rightarrow \beta$ that holds on R , α must be a **superkey** for R .

In simple terms: For any dependency, the determinant (the left-hand side) must be a candidate key or a superset of a candidate key.

BCNF Decomposition Algorithm

1. Let R be a relation that is not in BCNF.
2. Find a non-trivial FD $\alpha \rightarrow \beta$ that violates BCNF (i.e., α is not a superkey).
3. Decompose R into two relations:
 - $R_1 = \alpha \cup \beta$ (the attributes from the violating FD).
 - $R_2 = (R - \beta) \cup \alpha$ (all other attributes, plus the determinant).
4. Recursively check if R_1 and R_2 are in BCNF. If not, decompose them further.

BCNF Example 1: Staff Projects

Let's revisit our original relation:

$R(\underline{\text{staffID}}, \text{staffName}, \underline{\text{projectID}}, \text{projectName})$. The candidate key is $\{\text{staffID}, \text{projectID}\}$.

FDs:

1. $\text{staffID} \rightarrow \text{staffName}$
2. $\text{projectID} \rightarrow \text{projectName}$

Violation: staffID is not a superkey. So, $\text{staffID} \rightarrow \text{staffName}$ violates BCNF.

Decomposition:

- $R_1 = \{\text{staffID}, \text{staffName}\}$
- $R_2 = \{\text{staffID}, \text{projectID}, \text{projectName}\}$

Now, in R_2 , projectID is not a superkey. So, $\text{projectID} \rightarrow \text{projectName}$ violates BCNF.

BCNF Example 1: Staff Projects (Cont.)

$R(\underline{\text{staffID}}, \text{staffName}, \underline{\text{projectID}}, \text{projectName})$.

FDs:

1. $\text{staffID} \rightarrow \text{staffName}$
2. $\text{projectID} \rightarrow \text{projectName}$

Decomposition 1:

- $R_1 = \{\text{staffID}, \text{staffName}\}$
- $R_2 = \{\text{staffID}, \text{projectID}, \text{projectName}\}$

Decomposition of R_2 :

- $R_{2a} = \{\text{projectID}, \text{projectName}\}$
- $R_{2b} = \{\text{staffID}, \text{projectID}\}$

Final BCNF Schema: $(\text{staffID}, \text{staffName})$, $(\text{projectID}, \text{projectName})$, $(\text{staffID}, \text{projectID})$.

This is exactly the schema we arrived at intuitively!

BCNF Example 2: Student Courses

Consider $R(\underline{sid}, \underline{cid}, sname, cname, grade)$. The candidate key is $\{sid, cid\}$.

FDs:

1. $sid \rightarrow sname$
2. $cid \rightarrow cname$

Violation: sid is not a superkey. So, $sid \rightarrow sname$ violates BCNF.

Decomposition:

- $R_1 = \{sid, sname\}$
- $R_2 = \{sid, cid, cname, grade\}$

Now, in R_2 , cid is not a superkey. So, $cid \rightarrow cname$ violates BCNF.

BCNF Example 2: Student Courses (Cont.)

Consider $R(\underline{sid}, \underline{cid}, sname, cname, grade)$.

FDs:

1. $sid \rightarrow sname$
2. $cid \rightarrow cname$

Decomposition:

- $R_1 = \{sid, sname\}$
- $R_2 = \{sid, cid, cname, grade\}$

Decomposition of R_2 :

- $R_{2a} = \{cid, cname\}$
- $R_{2b} = \{sid, cid, grade\}$

Final BCNF Schema: $(sid, sname), (cid, cname), (sid, cid, grade)$.

Properties of Decomposition

Lossless vs. Lossy Decomposition

A decomposition is **lossless** if the original relation can be perfectly reconstructed by joining the decomposed tables. If the join produces spurious (extra, incorrect) tuples, the decomposition is **lossy**.

Rule for Lossless Join: A decomposition of R into R_1 and R_2 is lossless if and only if the set of common attributes ($R_1 \cap R_2$) is a superkey for either R_1 or R_2 .

Example: Decomposing $R(\text{staffID}, \text{staffName}, \text{projectName})$ into:

- $R_1(\text{staffID}, \text{staffName})$
- $R_2(\text{staffName}, \text{projectName})$

Common attribute: `staffName`, not a key for R_1 or R_2 .

If two staff members have the same name?

Sequence of Decomposition Matters

The final set of relations can differ.

Consider $R(\text{Branch}, \text{City}, \text{Customer}, \text{Loan})$

FDs:

- $\text{Branch} \rightarrow \text{City}$
- $\text{Loan} \rightarrow \text{Branch}$

The candidate key is $\{\text{Loan}, \text{Customer}\}$. Both FDs violate BCNF.

Path 1: Decompose on

$\text{Branch} \rightarrow \text{City}$

- $R_1(\text{Branch}, \text{City})$
- $R_2(\text{Loan}, \text{Branch})$
- $R_3(\text{Customer}, \text{Loan})$

Path 2: Decompose on

$\text{Loan} \rightarrow \text{Branch}$

- $R_1(\text{Loan}, \text{Branch})$
- $R_2(\text{Loan}, \text{City}, \text{Customer})$

Problems with BCNF: Dependency Preservation

BCNF guarantees a lossless join, but it does not always preserve all functional dependencies.

An FD is **preserved** if it can be checked on a single relation in the decomposed schema.

Example: Dependency Preservation

Consider $R(\text{sid}, \text{cid}, \text{instructor})$

FDs:

- $\{\text{sid}, \text{cid}\} \rightarrow \text{instructor}$ (A student in a course has one instructor)
- $\text{instructor} \rightarrow \text{cid}$ (An instructor teaches only one course)

Candidate keys are $\{\text{sid}, \text{cid}\}$ and $\{\text{sid}, \text{instructor}\}$.

Violation: $\text{instructor} \rightarrow \text{cid}$: instructor is not a superkey.

Decomposition:

- $R_1(\text{instructor}, \text{cid})$
- $R_2(\text{sid}, \text{instructor})$

The FD $\{\text{sid}, \text{cid}\} \rightarrow \text{instructor}$ is lost!

Can only be checked by joining the tables.

Thank You!
Questions?